Producers and Consumers

sending and receiving without losing messages

Watch the 4-min video

In one line: producers decide durability (acks, retries, idempotence, batching) and consumers decide correctness (when to commit, how many to fetch, how long processing may take); a safe default is acks=all with idempotence on the producer, and committing offsets only after processing on the consumer.

1 Two ways to lose data

Your producer sends a payment event and gets a success back. Then the broker leader crashes, and the event is gone. Your consumer, meanwhile, grabbed a thousand messages, auto-committed their offsets, and crashed halfway through; the other five hundred are never processed. Both sides have settings that decide whether messages survive.

2 Producer: durability

The **acks** setting says when the broker answers 'success'.

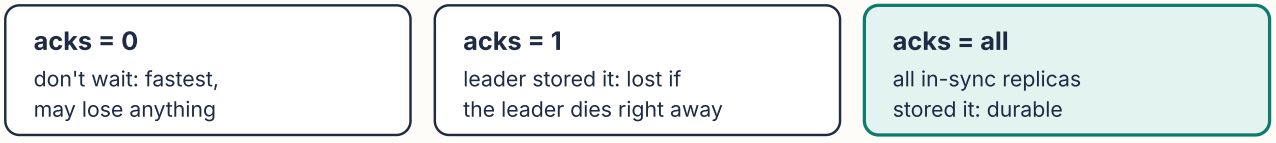


Figure 1. acks=all with min.insync.replicas = 2 (lesson 28) is the durable choice.

Retries handle transient errors, but a retry after a lost acknowledgement could write the message twice; **idempotence** makes the broker discard such duplicates using a producer ID and sequence numbers (default in recent Kafka clients). **Batching** (`linger.ms`, `batch.size`) and **compression** trade a few milliseconds of latency for much higher throughput. The **key** decides the partition, and so the ordering (lesson 29).

WORKED EXAMPLE: A PYTHON PRODUCER WITH AIOKAFKA

The payment service creates the producer with `acks='all'`, idempotence enabled, a 10 ms linger and lz4 compression, and sends each event keyed by order ID. `send_and_wait` returns only after all in-sync replicas have the event, so the API can safely tell the client the payment was recorded. If the leader dies a moment later, a follower that already has the event becomes leader. Throughput stays high because events from concurrent requests travel in shared batches.

```
producer = AIOKafkaProducer(
    bootstrap_servers=BROKERS,
    acks='all',                # wait for in-sync replicas
    enable_idempotence=True,   # no duplicates from retries
    linger_ms=10,              # batch for 10 ms
    compression_type='lz4')
await producer.send_and_wait('payments', value=event, key=order_id)
```

Figure 2. A durable producer configuration.

3 Consumer: correctness

A consumer **polls** batches and **commits offsets** to record progress. Auto-commit commits on a timer, possibly before processing finished: a crash then **loses** messages. The safe pattern: disable auto-commit, process the batch, then commit (at-least-once), and make processing idempotent so the occasional re-read is harmless (lesson 30).



Figure 3. Commit after processing. Duplicates are possible; losses are not.

Watch the time budget: if one poll's batch takes longer than `max.poll.interval.ms`, the broker assumes the consumer died and rebalances its partitions (lesson 27). Use a smaller `max.poll.records` or hand slow work to another pool.

COMMON MISTAKES

- acks=1 for payment data.
- Fire-and-forget sends without checking the result or using a callback.
- Auto-commit with slow or failing processing.
- Huge batches that exceed the poll interval and cause rebalances.

4 Key terms

acks: how many replicas must store a write

linger.ms / batch.size: how long and how much to batch

Auto-commit: periodic commit, possibly before processing

Idempotent producer: broker drops duplicate retries

Offset commit: recording the consumer's progress

max.poll.interval.ms: max time between polls before a rebalance

5 Interview questions

Q1. What does acks=all mean? BASIC

The leader replies success only after all in-sync replicas have the record, so it survives the loss of the leader.

Q2. Why commit offsets after processing? BASIC

If the consumer crashes after committing but before processing, those messages are skipped forever. Committing after gives at-least-once.

Q3. A payment event was acknowledged and later disappeared. What settings do you check? SCENARIO

Producer acks (should be all), topic replication factor and min.insync.replicas, and unclean leader election (should be off).

Q4. Consumers rebalance every few minutes under load. Why? SCENARIO

Processing a poll batch exceeds max.poll.interval.ms, so the group thinks the consumer died. Reduce max.poll.records, speed up processing, or increase the interval.

Q5. Why might enabling retries create duplicates, and how does Kafka prevent it? TRICKY

A retry after a lost ack rewrites a stored record. The idempotent producer attaches sequence numbers per partition so the broker drops the duplicate.

Q6. How does batching improve throughput? TRICKY

Many records share one request and one disk write, and compression works better on bigger batches, at the cost of a few milliseconds of latency.

Next: SD-I-004 · Topics, Partitions and Offsets backendroadmap.com